

TurboGator

Adam Suma

Frederico Aberle

Jannis Alsbach

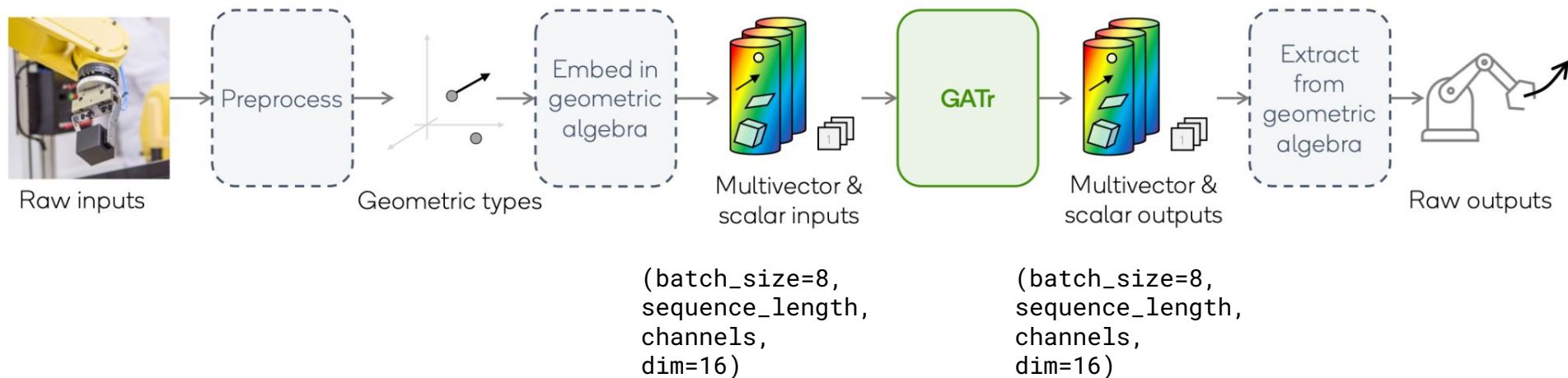
Mihai-George Licu



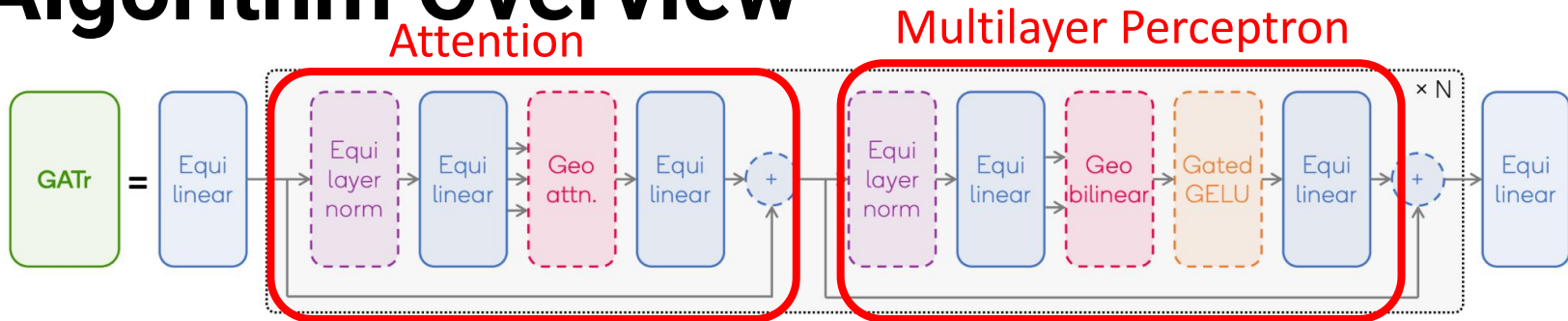
ETH

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

The Core Algorithm

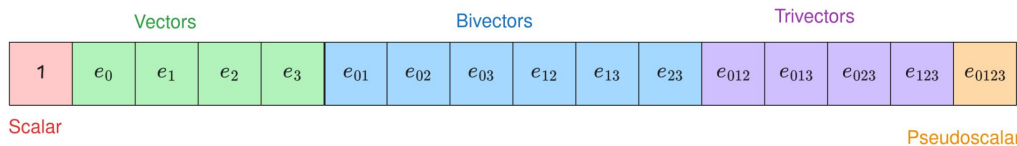


Algorithm Overview



- Initial embedding projection
- Transformer Blocks
 - MVOnlyGATrAttention
 - MVOnlyGATrMLP
- Out Projection

MVOnlyGATrModel(8, 32N, 2N, 16)

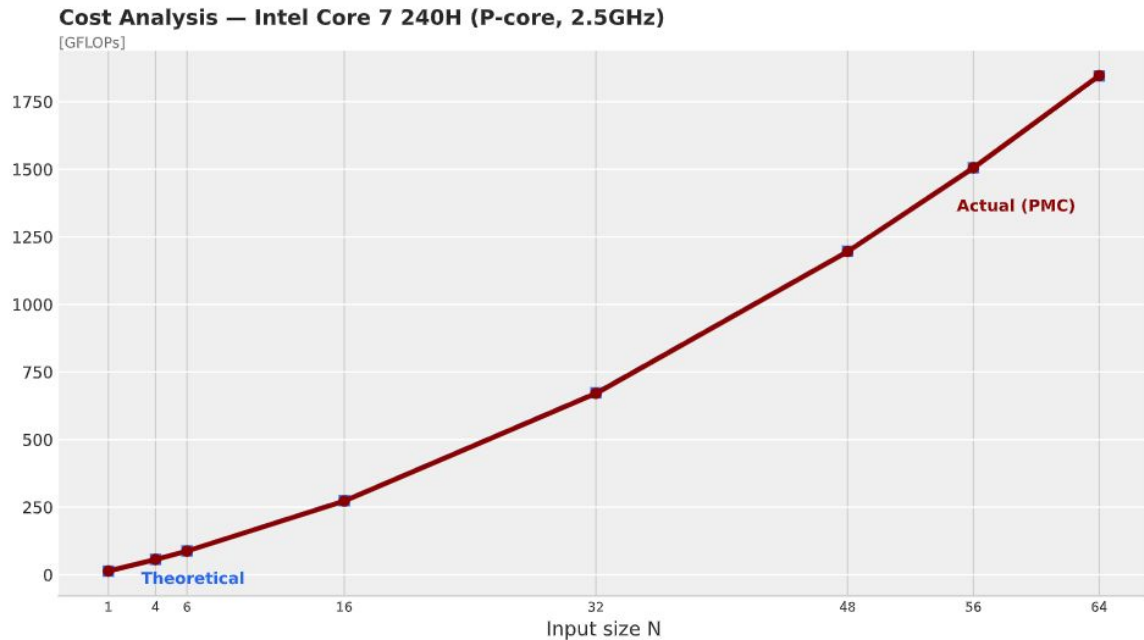


The Bottleneck: Dense Math on Sparse Data

- **Basis creates memory and computational overhead**
 - $16 \times 16 \times 16 = 4096$ floats allocated (mostly zero)
 - Multi-level looping
- **Introduction of new sparse data structure necessary**
 - Store non-zero coordinates with sign flips (~ 200 nnz)
 - Hardcode the output assignments

→ **Reduction of $\sim 95\%$ in data size**

Cost Analysis



Work (FLOPs) $\leq O(N^2)$

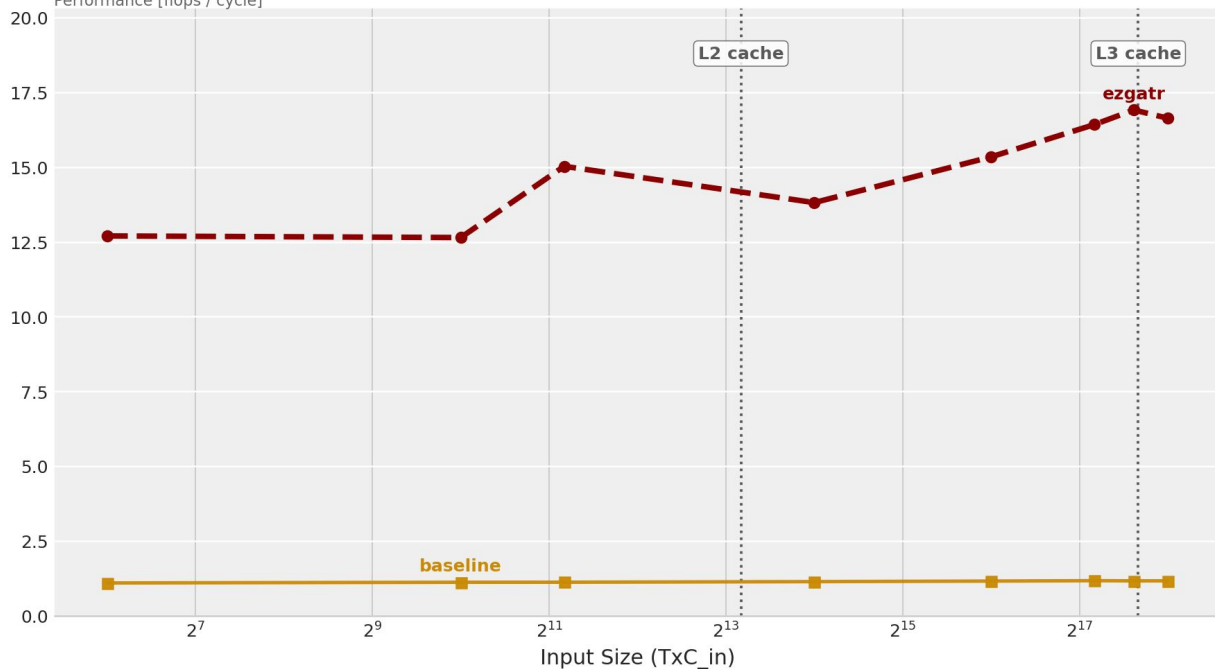
$$T = 32 * N$$

$$C_{in} = 2 * N$$

The naive C++ Baseline

Performance: Intel Core 7 240H (P-core, 2.5GHz)

Performance [flops / cycle]



2% of Theoretical Peak
(~0.8 / 40 FLOPs/cycle)

Optimized Versions

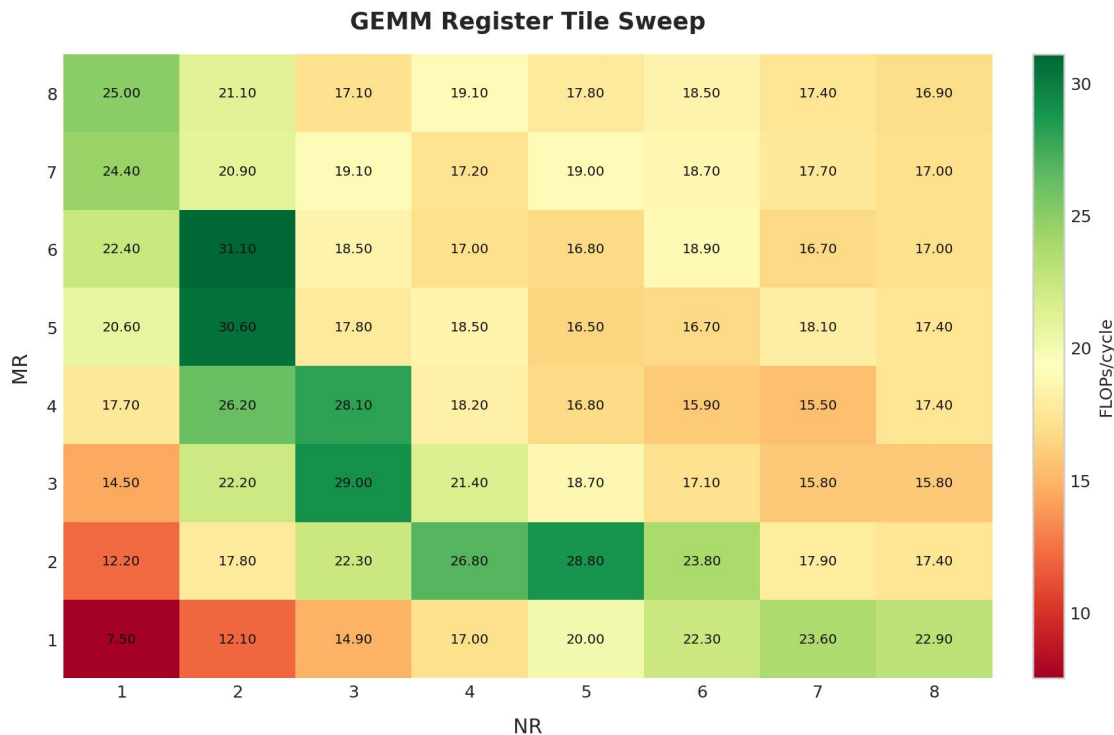
Incremental Improvements

- **Baseline**
 - Dense basis tensors, naive nested loops, basic contractions for equi_linear
- **ScalarGatorV1**
 - Initial Sparsity + Simplicity + Cache Blocking + Scalar replacement
- **ScalarGatorV2 (best scalar version)**
 - Online Softmax + Flash Attention + xtra Sparsity
- **TurboGator**
 - 256-bit AVX intrinsics with full pipelining
 - Glue layer improvements

Online Softmax & Flash Attention

- Classical (safe) Softmax:
 - 3 passes (find maximum, calculate exp+sum, normalize)
 - Online Softmax:
 - 2 streaming passes with running (max, sum) + on-the-fly correction
 - Flash Attention (enabled via online Softmax)
 - Tile-wise fused $S = QK^T \rightarrow P = \text{softmax}(S) \rightarrow O = P \cdot V$ with running (max, sum) correction across tiles
- $O(T \cdot d)$ memory traffic instead of $O(T^2)$

Optimal Tiling

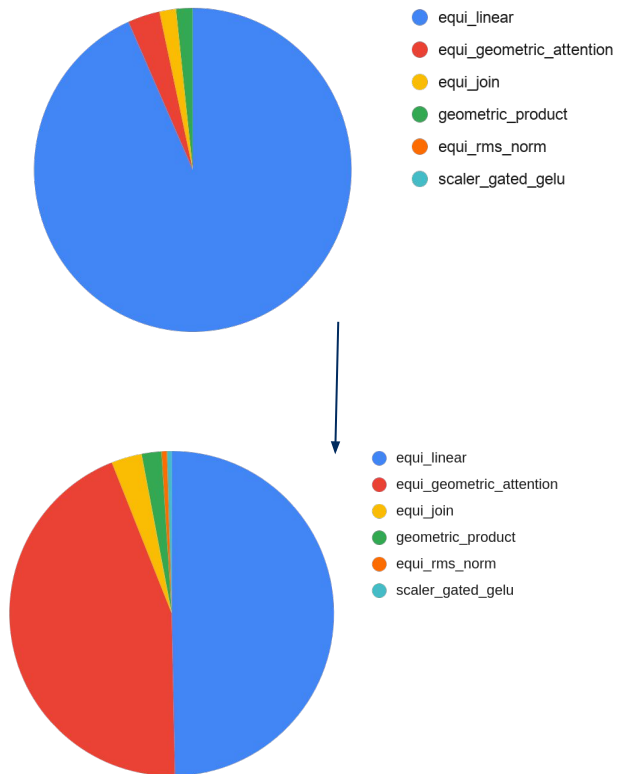


Best FLOPs/cycle performance: NR = 2, MR = 6 (31.10)

- Extracted optimum tiles for GEMM on our machine using a different simple matrix multiply program.
- For our specific task -> the optimal value was **4x2** because of divisibility by 32 of the specific input size.

Profiling

Distribution of Trace Time by Function



Baseline

equi_linear
149610ms

geom_attn
5226ms

160x

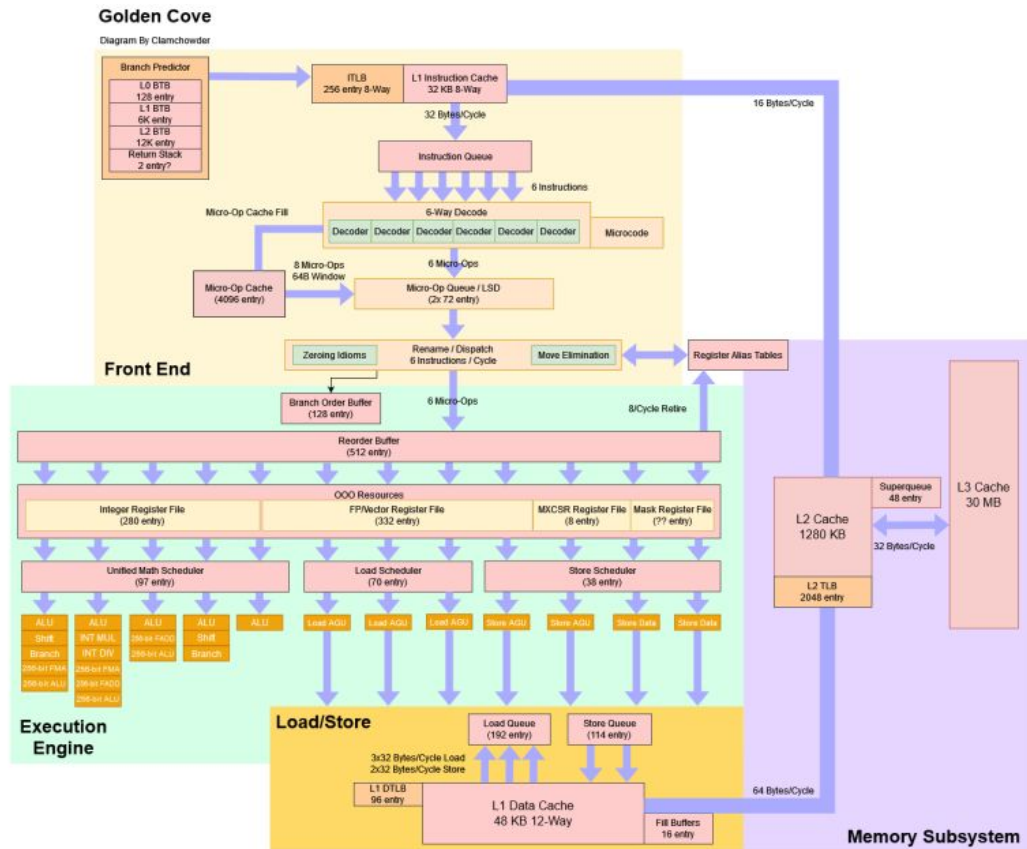
6.3x

TurboGator

932ms

831ms

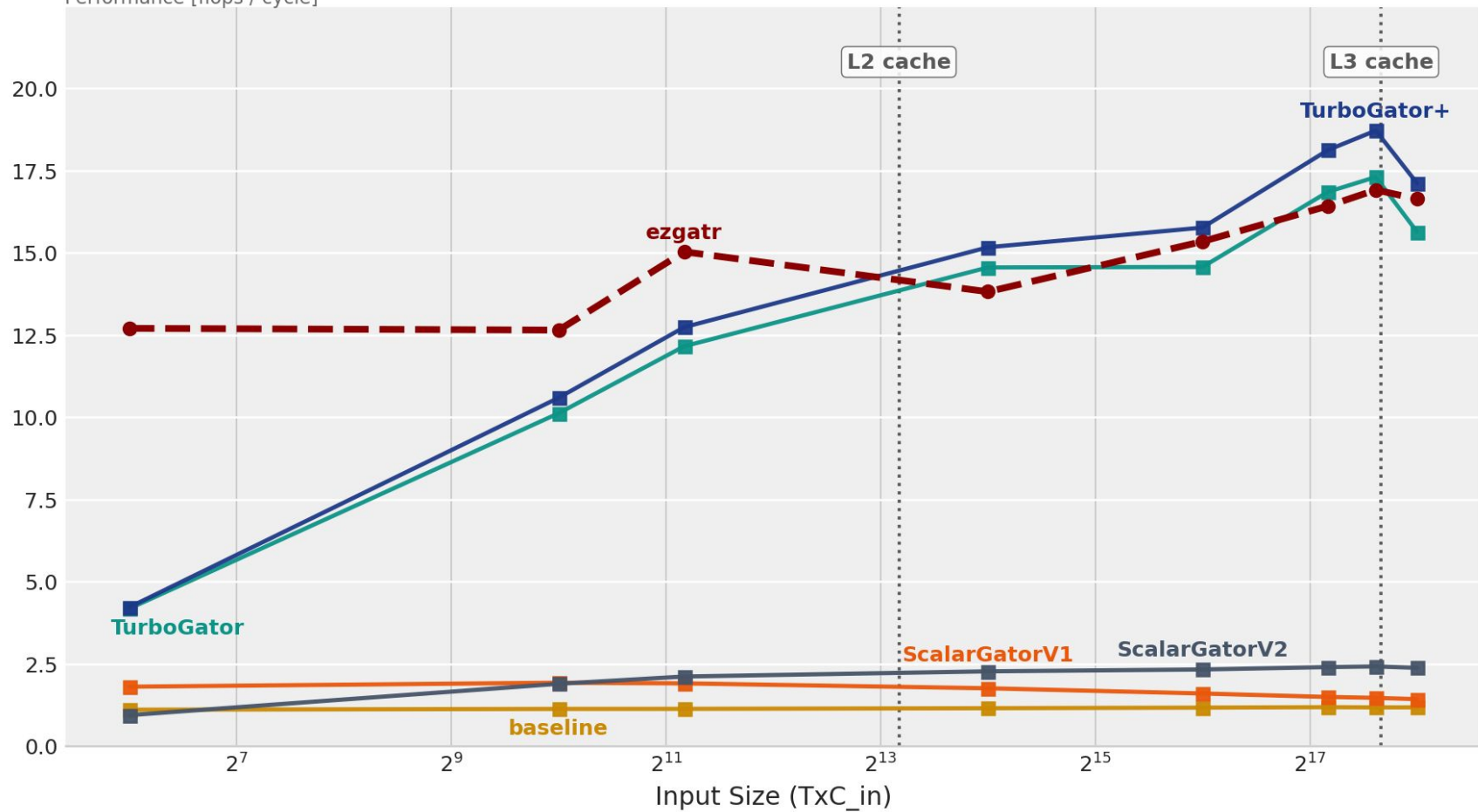
Server Setup



- Intel Core 7 240H Raptor Lake P-core (Golden Cove)
- Intel SSE4.1, SSE4.2, AVX2, FMA3
- TurboBoost off, pinned P-core 4 @ 2.5 GHz, 32GB LPDDR5
- $\pi_s = 5 \text{ flops/cycle}$
- $\pi_v = 40 \text{ flops/cycle}$
- $\beta (L1) = 96 \text{ bytes/cycle}$
- $\beta (L2) = 64 \text{ bytes/cycle}$
- $\beta (L3) = 32 \text{ bytes/cycle}$
- $\beta (\text{DRAM}) = 9 \text{ bytes/cycle}$
- L1d = 48 KiB, 12-way; Latency 4 cycles
- L2 = 1.25 MB; L3 = 24 MB (shared)

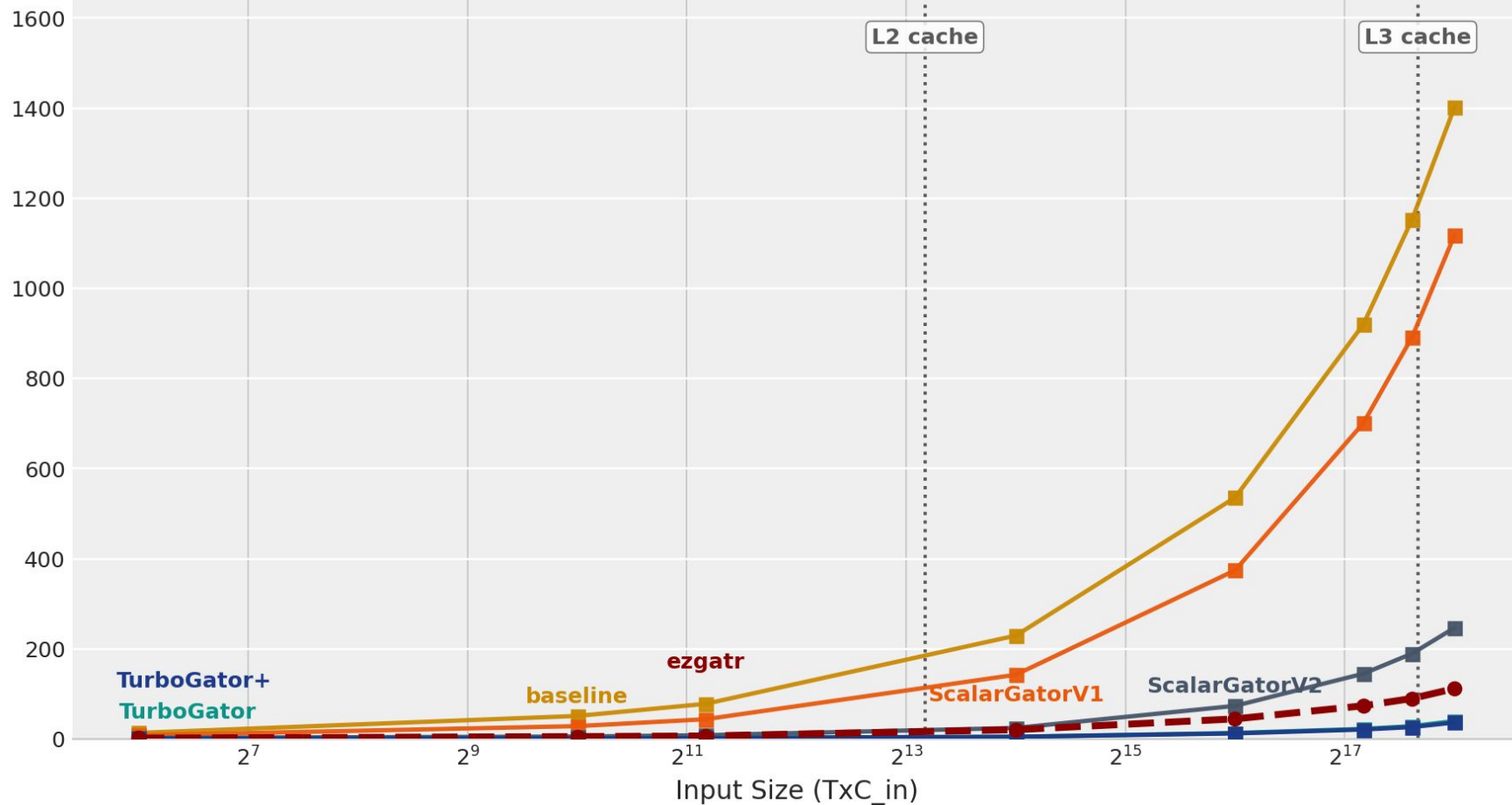
Performance: Intel Core 7 240H (P-core, 2.5GHz)

Performance [flops / cycle]



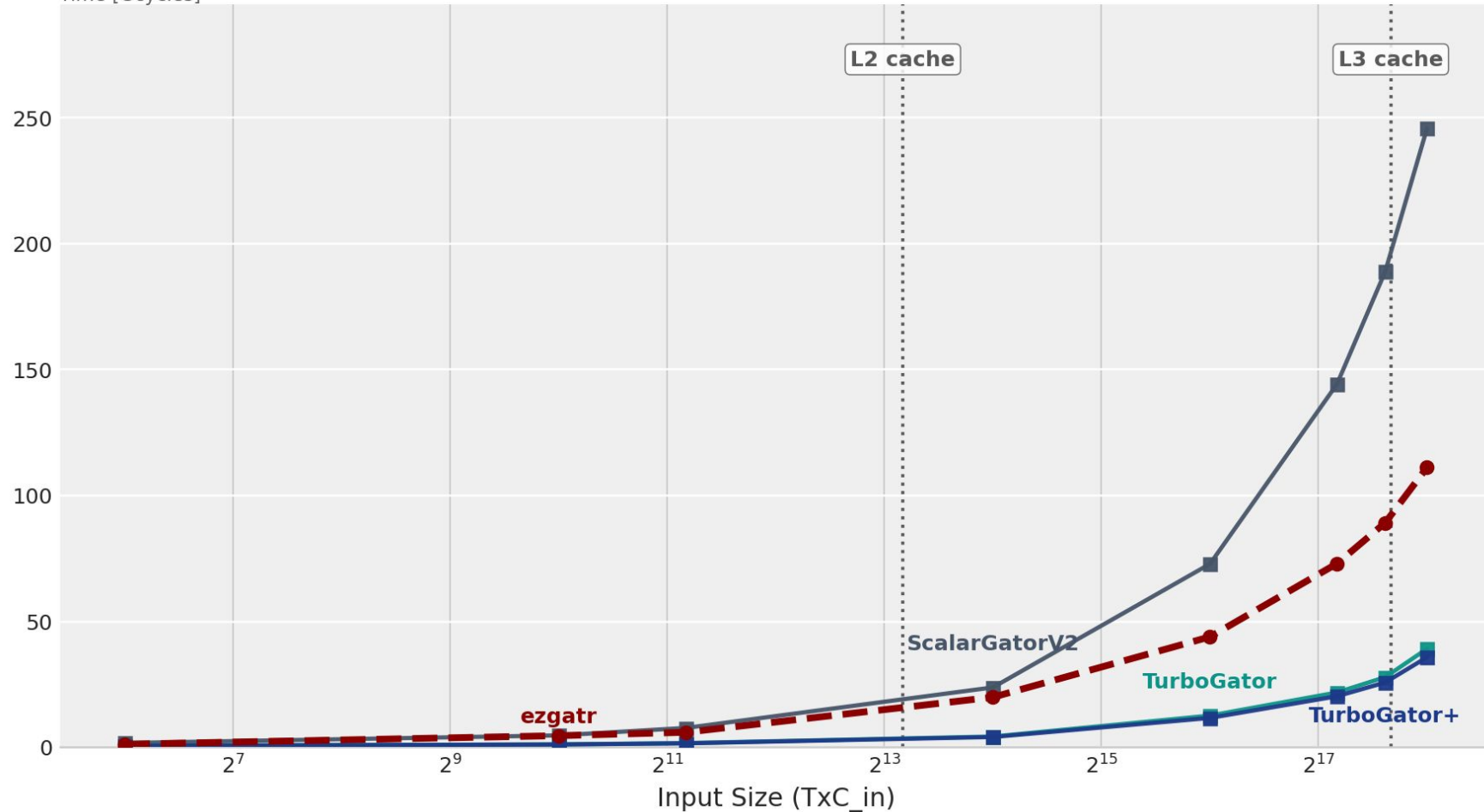
Runtime: Intel Core 7 240H (P-core, 2.5GHz)

Time [Gcycles]



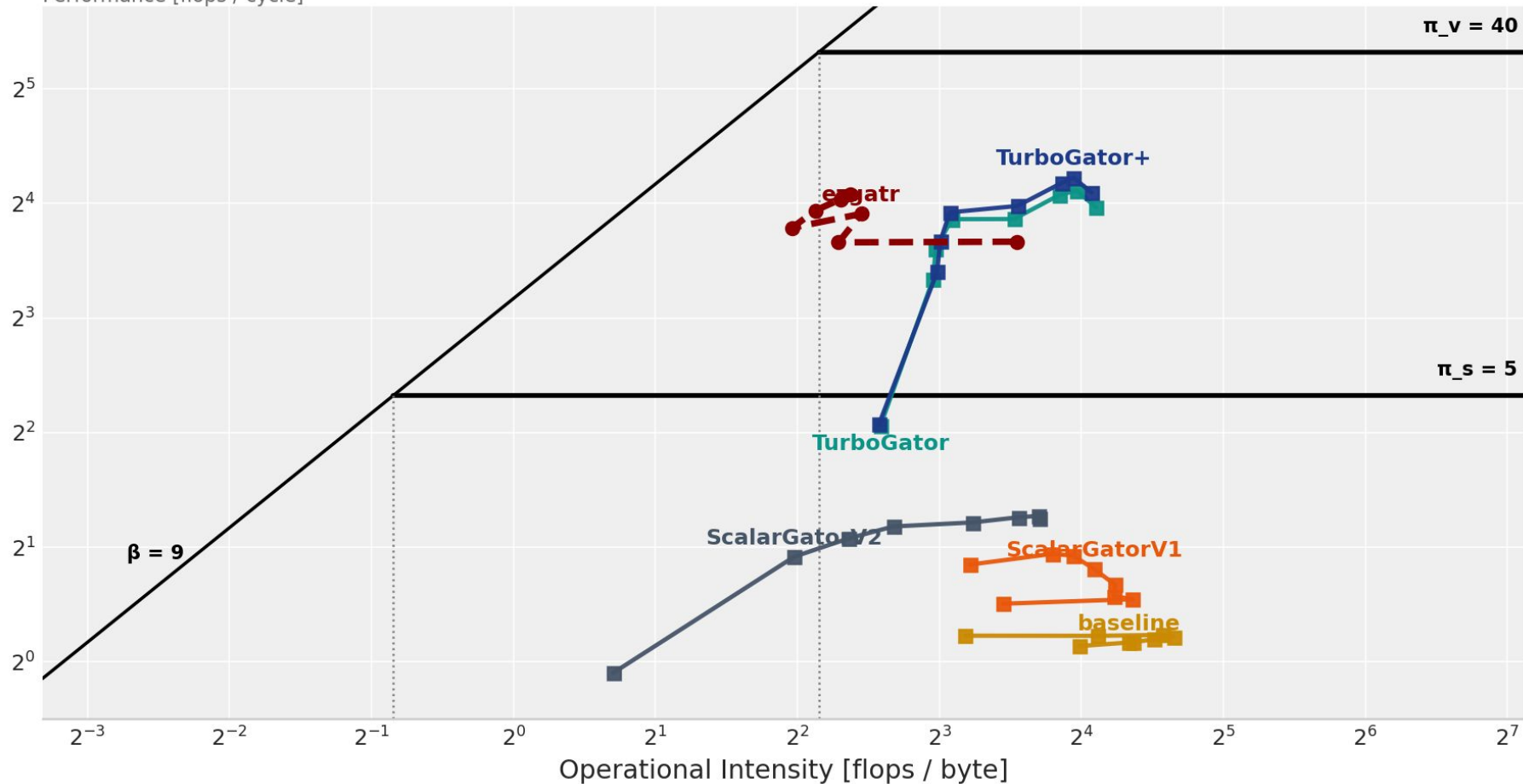
Runtime: Intel Core 7 240H (P-core, 2.5GHz) (Without Baseline)

Time [Gcycles]



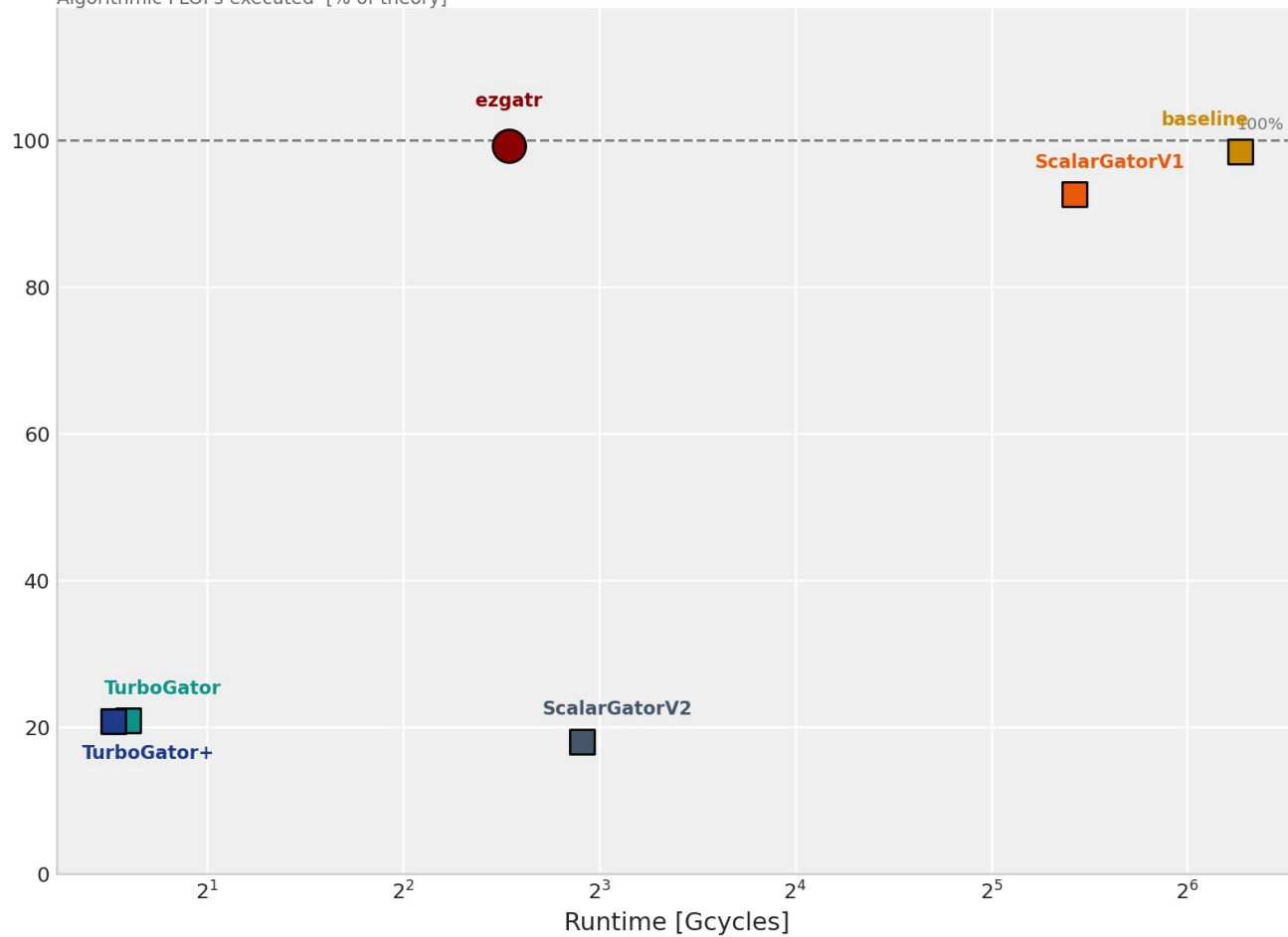
Roofline (DRAM): Intel Core 7 240H (P-core, 2.5GHz)

Performance [flops / cycle]



Work vs. Speed at N=6

Algorithmic FLOPs executed [% of theory]



Cycles speedup over ezgatr [x]

